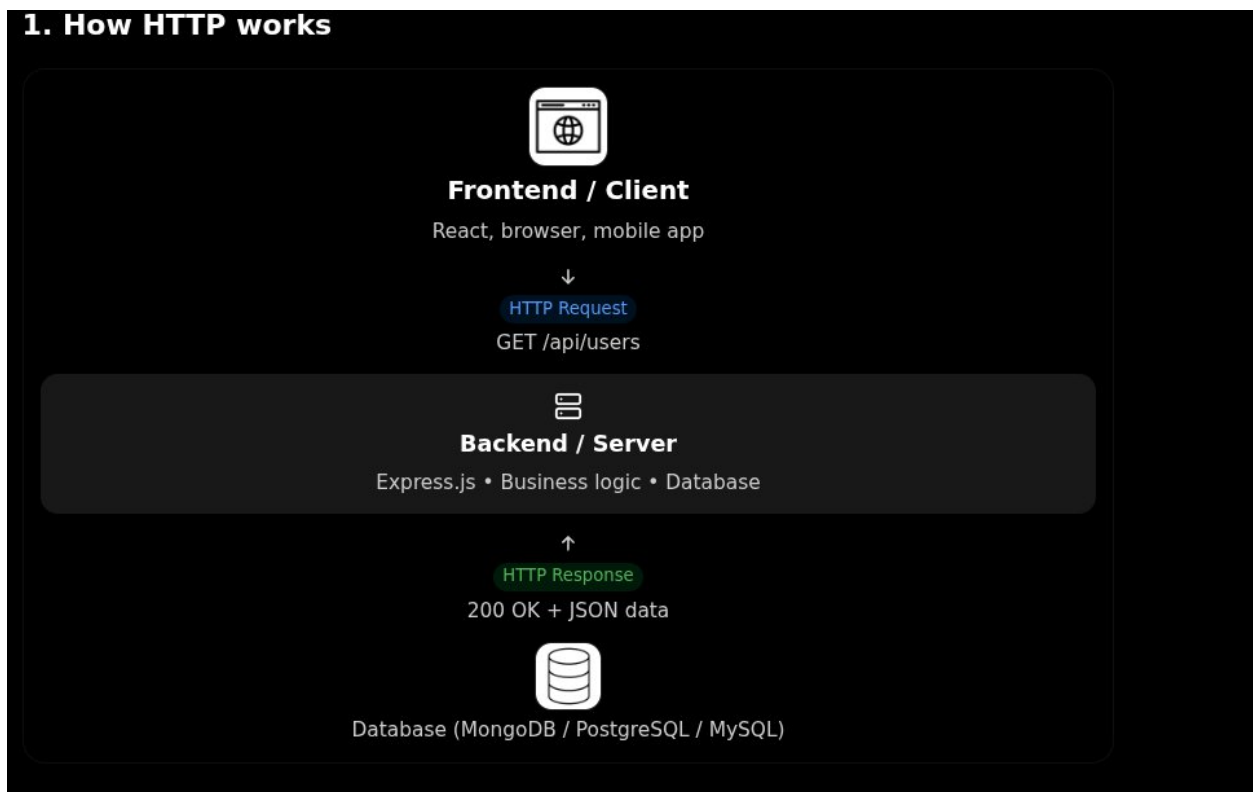


HTTP in Backend Development

HTTP (HyperText Transfer Protocol) is a communication protocol used between a client and a server to send and receive data over the internet.

In backend development, HTTP is used to create APIs, handle requests, and send responses.

1. How HTTP works



Example

React frontend sends:

GET /api/users

Express backend responds:

```
{  
  "success": true,  
  "users": [  
    {  
      "id": 1,  
      "name": "John",  
      "email": "john@example.com",  
      "password": "12345678",  
      "role": "user",  
      "status": "active",  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z",  
      "deleted_at": null  
    },  
    {  
      "id": 2,  
      "name": "Jane",  
      "email": "jane@example.com",  
      "password": "12345678",  
      "role": "user",  
      "status": "active",  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z",  
      "deleted_at": null  
    },  
    {  
      "id": 3,  
      "name": "Admin",  
      "email": "admin@example.com",  
      "password": "12345678",  
      "role": "admin",  
      "status": "active",  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z",  
      "deleted_at": null  
    }  
  ]  
}
```

```

    {
      "name": "Uttam",
      "role": "Backend Developer"
    }
  ]
}

```

2. HTTP Methods

HTTP methods define what action the backend should perform.

Method	Purpose	Example
GET	Read data	Get all users
POST	Create data	Register user
PUT	Replace data	Update full profile
PATCH	Partially update	Change username
DELETE	Delete data	Remove user

Express.js example

```

import express from "express";

const app = express();

app.use(express.json());

// GET
app.get("/api/users", (req, res) => {
  res.json({ message: "Get users" });
});

// POST
app.post("/api/users", (req, res) => {

```

```

    res.status(201).json({
      message: "User created",
      data: req.body
    });
  });

// PUT
app.put("/api/users/:id", (req, res) => {
  res.json({ message: "User replaced" });
});

// PATCH
app.patch("/api/users/:id", (req, res) => {
  res.json({ message: "User updated" });
});

// DELETE
app.delete("/api/users/:id", (req, res) => {
  res.json({ message: "User deleted" });
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});

```

3. HTTP Request

A request is sent from the client to the backend.

POST /api/users HTTP/1.1

Host: localhost:5000

Content-Type: application/json

Authorization: Bearer token

```
{  
  "name": "Uttam",  
  "email": "uttam@example.com"  
}
```

Main parts

1. Method: POST
2. URL / Path: /api/users
3. Headers: Content-Type, Authorization
4. Body: Data sent to the server

4. HTTP Response

The backend sends a response to the client.

HTTP/1.1 201 Created

Content-Type: application/json

```
{  
  "message": "User created successfully"  
}
```

Main parts

5. Status code
6. Response headers
7. Response body

5. HTTP Status Codes

Code	Meaning
200	OK — Request successful
201	Created — Resource created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found

500 Internal Server Error

Example:

```
app.get("/api/profile", (req, res) => {  
  res.status(200).json({  
    message: "Profile fetched"  
  });  
});
```

6. HTTP vs HTTPS

HTTP

Data is not encrypted by HTTP itself

Uses port 80 by default

Less secure on untrusted networks

Example: `http://localhost:5000`

HTTPS

Data is encrypted using TLS

Uses port 443 by default

Protects data in transit

Example: `https://example.com`

For production backend APIs, use HTTPS. Local development commonly uses HTTP.

7. HTTP vs WebSocket

Since you are learning backend with Node.js, this is important.

HTTP

Request-response communication

Client sends request

Common for REST APIs

Example: Fetch users

Can use repeated polling

WebSocket

Persistent, two-way communication

Server can send messages anytime

Common for real-time apps

Example: Live chat

Supports real-time events

HTTP

Client → Request → Server

Client ← Response ← Server

WebSocket

Client ↔ Persistent Connection ↔ Server

Simple Request vs Preflighted Request in HTTP

In backend development, especially when working with Express.js, React, and CORS, browsers sometimes send an HTTP request directly and sometimes send a preflight request first.

1. Simple Request

A simple request is a cross-origin HTTP request that meets the browser's conditions for being sent without a CORS preflight (OPTIONS) request.

Example

Frontend (React) sends a request to the backend:

```
fetch("http://localhost:5000/api/users", {  
  method: "GET"  
});
```

The browser directly sends:

```
GET /api/users HTTP/1.1
```

```
Host: localhost:5000
```

```
Origin: http://localhost:5173
```

Backend responds:

```
HTTP/1.1 200 OK
```

```
Access-Control-Allow-Origin: http://localhost:5173
```

```
Content-Type: application/json
```

```
{"message": "Users fetched"}
```

Conditions for a simple request

1. Method is GET, HEAD, or POST.
2. Request headers are limited to CORS-safelisted headers.
3. If using Content-Type, it must be one of:
 - o text/plain
 - o application/x-www-form-urlencoded
 - o multipart/form-data
4. It must not contain disallowed custom headers.

Important: A simple request can still be blocked from reading the response if the backend does not return the correct CORS headers.

2. Preflighted Request

A preflighted request is a cross-origin request where the browser first sends an OPTIONS request to ask the backend whether the actual request is allowed.

It is used when the request does not meet the conditions for a simple request.

Example: JSON POST request

React frontend:

```
fetch("http://localhost:5000/api/users", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({  
    name: "Uttam"  
  })  
});
```

Because application/json is not a CORS-safelisted Content-Type, the browser may perform a preflight.

Step 1: Preflight request

Browser sends:

OPTIONS /api/users HTTP/1.1

Host: localhost:5000

Origin: http://localhost:5173

Access-Control-Request-Method: POST

Access-Control-Request-Headers: content-type

Step 2: Backend response

HTTP/1.1 204 No Content

Access-Control-Allow-Origin: http://localhost:5173

Access-Control-Allow-Methods: POST, GET, OPTIONS

Access-Control-Allow-Headers: Content-Type

Step 3: Actual request

If the preflight succeeds, the browser sends:

POST /api/users HTTP/1.1

Host: localhost:5000

Origin: http://localhost:5173

Content-Type: application/json

```
{"name": "Uttam"}
```

The backend processes the request and sends the response.

3. Main Differences

Feature	Simple Request	Preflighted Request
Preflight OPTIONS	Not sent	Sent first
Actual request	Sent directly	Sent after preflight approval
Common methods	GET, HEAD, POST	PUT, PATCH, DELETE, or non-simple POST
JSON Content-Type	Not simple	Usually triggers preflight
Custom Authorization header	Usually triggers preflight	Common reason

Browser purpose

Send request under simple
CORS rules

Check permission before sending
actual request

4. Visual Flow

Simple Request:

React Frontend



GET Request



Express Backend

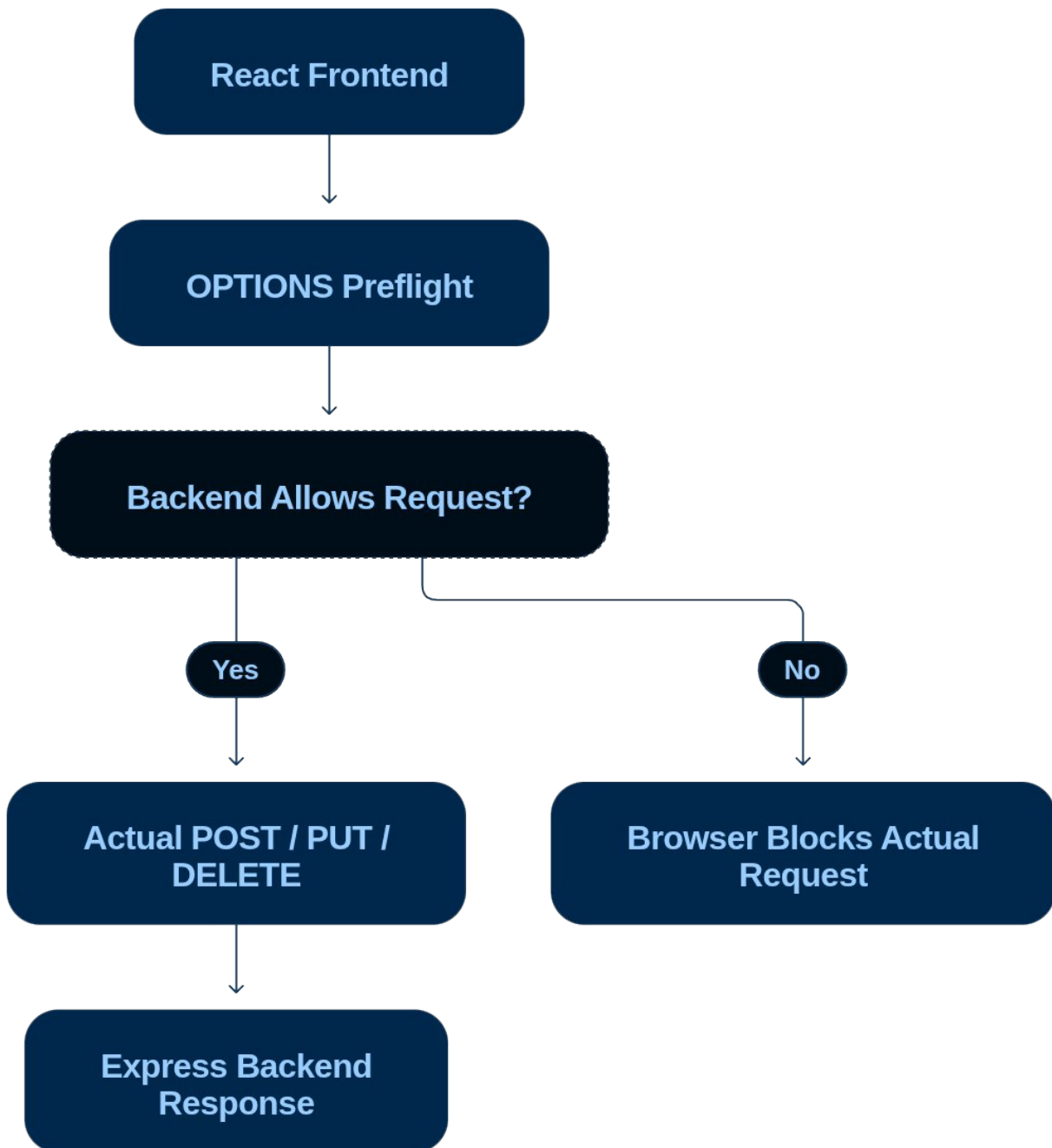


**Response with CORS
Headers**



Browser Allows Response

Preflighted Request:



5. Express.js CORS Example

Install:

```
npm install cors
```

Backend:

```
import express from "express";
import cors from "cors";

const app = express();

app.use(
  cors({
    origin: "http://localhost:5173",
    methods: ["GET", "POST", "PUT", "PATCH", "DELETE"],
    allowedHeaders: ["Content-Type", "Authorization"]
  })
);

app.use(express.json());

app.post("/api/users", (req, res) => {
  res.json({
    message: "User created",
    data: req.body
  });
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

The cors middleware handles the CORS headers and preflight requests for the configured routes.

